

Progressive Web Apps

Von der Webanwendung zur installierbaren, offlinefähigen App

Web Technologien 2026 · Workshop

Dienstag, 21. Juli 2026 · ca. 120 Minuten

Manuel Stamm

Agenda – Theorie & Praxis im Wechsel

Block	Theorie	Danach direkt
A	PWA-Idee & Praxisbeispiele (~8 Min.)	Setup / Demo
B	Manifest & Install (~8 Min.)	Aufgabe 1 (~20 Min.)
C	Service Worker & Precache (~10 Min.)	Aufgabe 2 (~25 Min.)
D	Caching-Strategien (~8 Min.)	Aufgabe 3 (~20 Min.)
E	DevTools, Lighthouse, Kamera (~5 Min.)	Aufgabe 4 gemeinsam (~10 Min.)
—	Abschluss & Diskussion	~5–10 Min.

Lernziele

Nach dem Workshop könnt ihr:

- erklären, was eine PWA von einer klassischen Webanwendung unterscheidet
- ein gültiges Web App Manifest einbinden und Installierbarkeit prüfen
- einen Service Worker registrieren und Assets precachen
- mit DevTools, Cache Storage und Lighthouse analysieren
- Caching-Strategien (Cache-First vs. Network-First) begründen

Setup & Demo: FocusBoard

- Starter unter app/ – bewusst ohne Manifest & Service Worker
- npm start → <http://localhost:3000>
- Referenzlösung (erst nach den Aufgaben): npm run start:loesung → :3001
- Aufgaben: aufgaben/01 ... 04 · Lösungen: Lösungsbranches

Block A · Was ist eine PWA?

PWA: Reichweite des Webs + App-Fähigkeiten

- installierbar (Homescreen, Startmenü, App-Ordner)
- robuster unter instabilen Netzbedingungen
- app-ähnliche UX (z. B. standalone)

*„PWAs offer similar capabilities to mobile and desktop apps, they are reliable even in unstable network conditions, and are installable...”
— web.dev, What does it take to be installable? [3]*

Praxisbeispiele (Case Studies)

Beispiel	Ergebnis (laut Case Study)	Q
Twitter Lite	mehr Engagement, weniger Daten vs. Native-Download	[16]
MakeMyTrip	Conversion Mobile Web 3x; Sessions +160%	[17]
Goibibo	Conversion +60% vs. altem Mobile-Web-Flow	[18]
Trivago	nach Offline: 67% setzten die Session fort	[19]

Motive: Performance, Datenverbrauch, Zuverlässigkeit, Reichweite ohne Store-Zwang.

Case Studies – Details

- Twitter Lite: ~600 KB vs. ~23,5 MB native Android-App; SW schrittweise [16]
- MakeMyTrip: Conversion ×3; Page-Load –38%; Bounce –20% [17]
- Goibibo: +60% Conversion vs. altem Web-Flow; Share & Co. als Funnel-Hebel [18]
- Takeaway: Caching und schlanke Auslieferung sind Business-relevant

Abgrenzung & Bausteine

	Website	PWA (typisch)	Native
Verteilung	URL	URL + optional Installation	Store
Offline	meist nicht	SW + Cache möglich	oft ja
Updates	sofort	SW-Update-Zyklus	Store

Bausteine: Manifest · Service Worker (Offline; in Chrome nicht mehr Pflicht für Install) · Secure Context.
„PWA“ = Muster aus Standards – kein einzelnes API.

Block B · Manifest & Installierbarkeit

Web App Manifest

- JSON: wie die App erscheinen und starten soll [5][6]
- Empfohlene Endung: .webmanifest (häufig auch manifest.json) [6][7]

```
<link rel="manifest" href="./manifest.webmanifest" />
```

Wichtige Manifest-Felder

Feld	Rolle
name / short_name	Anzeigename (mind. eines in Chromium)
icons	mind. 192×192 und 512×512
start_url	Startadresse der installierten App
display	z. B. standalone, fullscreen, minimal-ui
theme_color / background_color	UI-/Splash-Farben
scope	welche URLs zur App gehören

Chrome: Install-Promotion & display

- noch nicht installiert
- Engagement: ≥ 1 Klick/Tap + $\geq \sim 30$ s Besuch
- HTTPS (lokal: localhost)
- gültiges Manifest mit Pflichtfeldern
- display: standalone $\rightarrow$ app-ähnlich ohne Adressleiste (Workshop-Empfehlung)

JETZT HANDS-ON

Jetzt: Aufgabe 1

Manifest & Installierbarkeit · ca. 20 Min.

- app/manifest.webmanifest anlegen
- In index.html verlinken
- Application → Manifest prüfen
- Install-UI / Installierbarkeit testen
- Danach: Sync – was seht ihr im Manifest-Panel?

→ [aufgaben/01-manifest.md](#)

Block C · Service Worker

Was ist ein Service Worker?

- Ereignisgesteuertes Worker-Skript (Origin + Scope)
- Requests abfangen (fetch), Cache nutzen, Offline steuern
- Secure Context: HTTPS oder localhost – nicht file://

```
if ("serviceWorker" in navigator) {  
  navigator.serviceWorker.register("./sw.js");  
}
```

Lebenszyklus & Precaching

register() → installing → installed → activating → activated
Update: neuer SW oft „waiting“

Event	Typische Aufgabe
install	App-Shell precachen (caches.open + addAll)
activate	Alte Caches löschen
fetch	Strategien (nächster Block)

skipWaiting() / clients.claim() steuern die Übernahme. [10]

Update-Fallen

- Update-Check u. a. bei Navigation im Scope [10]
- Neuer SW parallel; oft Warten bis alte Clients weg sind
- Byte-gleiche sw.js → kein Update; Inhalt ändern oder Cache-Namen versionieren
- DevTools: Application → Service Workers → „Update on reload“ [11]

JETZT HANDS-ON

Jetzt: Aufgabe 2

Service Worker & Precaching · ca. 25 Min.

- app/sw.js mit install / activate
- App-Shell precachen
- SW in der Seite registrieren
- Application: Worker + Cache Storage prüfen
- Offline-Reload der Shell testen

→ [aufgaben/02-service-worker.md](#)

Block D · Caching-Strategien

Cache Storage & Strategien

Strategie	Idee	Typisch für
Cache-First	Cache, sonst Netz	App-Shell
Network-First	Netz, sonst Cache	frische Daten

Offline ≠ alles lokal – nur Cache / localStorage. [12]

Cache-First vs. Network-First

- Cache-First: schnell/offline-robust, Risiko veraltet [12][14]
- Network-First: frisch online; offline nur mit vorherigem Cache [12]
- FocusBoard Shell → Cache-First; tips.json → Network-First; Aufgaben → localStorage

Grenzen von PWAs (kurz)

- Uncached Requests scheitern offline [12]
- Browser-/Plattform-Unterschiede bei Install & Hintergrund-APIs [7]
- Push / Background Sync: eigene APIs, Support prüfen [9][14]
- Große Medien brauchen bewusste Cache-Policies

JETZT HANDS-ON

Jetzt: Aufgabe 3

Caching-Strategien · ca. 20 Min.

- Cache-Namen erhöhen (z. B. v2)
- Shell → Cache-First
- tips.json → Network-First
- Online/Offline experimentieren
- Danach: Sync – welche Strategie für den Tipp, und warum?

→ [aufgaben/03-caching.md](#)

Block E · Tooling & Berechtigungen

DevTools & Lighthouse

- Application: Manifest, Service Workers, Cache Storage, Clear storage [11]
- Lighthouse: PWA-/Install-Checks (UI je nach Version) [4][15]
- Seit Lighthouse 11: SW-Check entfernt – Install ohne SW möglich; Offline braucht SW trotzdem [4][9]
- Praxis: Audit gegen `http://localhost:3000`

Kamera-Berechtigung (Live-Demo)

- FocusBoard: Kamera-Spiegel mit `getUserMedia({ video: true })` [20]
- Browser-Permission-Prompt; Status via Permissions API (Support variiert) [21]
- Secure Context nötig (wie Service Worker) [7][20]
- Stream nur lokal (`<video> + scaleX(-1)`)
- Workshop: starten → Prompt → Site-Settings widerrufen

JETZT HANDS-ON

Jetzt: Aufgabe 4 (gemeinsam)

DevTools, Lighthouse & Kamera · ca. 10 Min.

- Application-Panel durchgehen
- Offline + Network prüfen
- Kurzer Lighthouse-Lauf
- Optional: Kamera starten / Berechtigung widerrufen

→ [aufgaben/04-devtools-lighthouse.md](#)

Abschluss

Diskussionsfragen

- Welche Install-Voraussetzungen erfüllt eure App – welche nur optional? [3][7]
- Welche Caching-Entscheidung würdet ihr im Produktivprojekt anders treffen? [12]
- Welcher DevTools-/Lighthouse-/Permission-Fund hat euch überrascht? [11][4][20]

Quellen (1/2)

#	Quelle	URL
[1]	MDN – Progressive web apps	developer.mozilla.org/docs/Web/Progressive_web_apps
[2]	web.dev – Learn PWA	web.dev/learn/pwa
[3]	web.dev – Install criteria	web.dev/articles/install-criteria
[4]	Chrome – Lighthouse 11	developer.chrome.com/blog/lighthouse-11-0
[5]	web.dev – Web app manifest	web.dev/learn/pwa/web-app-manifest
[6]	web.dev – Add a manifest	web.dev/articles/add-manifest
[7]	MDN – Making PWAs installable	.../Guides/Making_PWAs_installable
[8]	MDN – Service Worker API	.../Web/API/Service_Worker_API
[9]	web.dev – Service workers	web.dev/learn/pwa/service-workers
[10]	Chrome/Workbox – SW lifecycle	.../workbox/service-worker-lifecycle
[11]	Chrome DevTools – Debug PWAs	.../devtools/progressive-web-apps
[12]	MDN – Caching (PWA)	.../Guides/Caching
[13]	MDN – fetch event	.../fetch_event
[14]	MDN – Offline & background	.../Offline_and_background_operation
[15]	Lighthouse – SW audit	.../lighthouse/pwa/service-worker
[16]	web.dev – Twitter Lite	web.dev/case-studies/twitter
[17]	web.dev – MakeMyTrip	web.dev/case-studies/make-my-trip

Quellen (2/2)

#	Quelle	URL
[18]	web.dev – Goibibo	web.dev/case-studies/goibibo
[19]	web.dev – business success (Trivago)	web.dev/articles/drive-business-success
[20]	MDN – getUserMedia	.../MediaDevices/getUserMedia
[21]	MDN – Permissions API	.../Web/API/Permissions_API

Vielen Dank für die Aufmerksamkeit!

Repo: github.com/Manusta01/webtech2026-workshop-pwa

Fragen?